

Concurrent Task Dispatcher

Chris Sanchez - 3334-01-Spring2026

Threads, Components, and Shared Data

There are four main parts to this system. Using a fixed random seed, the generator thread generates jobs, gives each one a kind, duration, and arrival time, and then transmits them one at a time into an MPSC channel with an adjustable arrival gap. When it's done, it immediately closes the channels by dropping its ender. After receiving tasks from that channel, the dispatcher thread will call the pool, increase a shared AtomicU64 Counter, and slap each task with a dispatch time. carry out to complete the task. When the generating channel ends, it will then leave. Eight designated worker threads, each looping on a common bounded sync_channel with a 100 capacity limit, are stored in the thread pool. After completing the tasks, workers send a CompletionRecord back to the main thread using either a busy counter loop for CPU tasks or thread::sleep for IO tasks. In order to track the minimal number of available workers seen during the run—which is shown in the summary—a monitor thread runs alongside the dispatcher, sampling the active_workers atomic every 5 ms. The dispatcher and generator are joined by the main thread, which prints the summary and drains the completion channel for the precise number of counter reports that have been sent. When the pool is dropped, all eight threads are cleanly joined and each worker receives a termination notice.

Additionally, the threads share three pieces of state. To enable all eight workers to pull from the same channel, the pool receiver is wrapped into Arc<Mutex<Receiver<Message>>>. Workers are operating concurrently because mutex is only kept during recv(), not during execution. The dispatcher and main thread share an Arc<AtomicU64> called submitted_count. Since only one increment and one load are required across the join boundary, an atomic is adequate. An Arc<AtomicUsize> called active_workers was introduced as a third variable. It is increased when a worker takes on a task and decreased when it is completed. This will eliminate the need for extra locks and enable the monitor thread to view the pool pressure in real time.

Channels and why they were chosen

There are three channels in use. Task structs are transported from the generator to the dispatcher via an unbounded mpsc channel, which is unbounded because tasks come at a reasonable rate and won't expand indefinitely. A 100-capacity constrained sync channel carries the bound creates backpressure so that the dispatcher cannot queue an infinite number of jobs ahead of the workers and use an infinite amount of memory by sending message values from the dispatcher to the workers inside the pool. CompletionRecord structs from all workers are carried

back to the main thread via a second unbounded mpsc channel, which automatically closes when each sender clone is discarded, indicating to the main thread that collection is finished. Channels were chosen over shared collections because they convey data ownership instead of sharing it, avoiding the need for extra locks and fits Rust's ownership model naturally.

Scheduling Policy and Trade-offs

FIFO is the policy. Workers receive tasks in the same order that they are generated and sent across both channels. There is no prioritizing or sorting. Since both channels are arranged by design, FIFO was selected since it naturally fits the architecture and doesn't require decision logic from the dispatcher. Because no task is omitted in favor of another and the work is divided equally among the eight employees, it offers high fairness. With an average wait time of 119 milliseconds and an average turnaround time of 141 milliseconds, worker utilization was consistent throughout the pool in Experiment A with a balanced 50/50 CPU and IO workload of 500 jobs.

Head-of-line blocking is FIFO's flaw. No matter how quickly the shorter activities would finish, a lengthy CPU task at the front of the channel makes them wait. Experiment B demonstrated this by using 80% CPU jobs with burst arrivals and durations of up to 80 milliseconds. The maximum wait time was 692 milliseconds, while the average wait time was 455 milliseconds, almost four times longer than in Experiment A. The average turnaround increased to 763 milliseconds, more than five times the 141 millisecond estimate from Experiment A. Because they were trapped behind CPU tasks, IO tasks as brief as two milliseconds accrued wait periods that were several times longer than their execution time. This was made worse by the sync_channel capacity of 100, which prevented the dispatcher while workers slowly drained long tasks.

Bugs Encountered and Fixed

Following both experiments, worker utilization showed values over 1300%. The accumulation line increased each worker's busy time by `turnaround_ms` minus `wait_ms`. The issue is because `wait_ms` only measures `dispatch_time` to execution start, but `turnaround_ms` measures `arrival_time` to completion. Every record has excess time that inflates busy totals well beyond the makespan since the time between arrival and dispatch is never deducted. Adding a `duration_ms` field to the `CompletionRecord` set straight from the job is the proper solution. Instead, add `duration_ms` inside the worker closure. Therefore, because of scheduling delays, timestamp gaps, `duration_ms`, resulting in percentages numbers ranging between 0 and 100 percent, is always accurate because it is the precise simulated execution time.

Remaining Starvation and Unfairness

There are still two causes of practical unfairness. First, although quick IO jobs accumulate in the channel, a spike in lengthy CPU tasks can take up all eight workers. Although the IO jobs are not always starved, FIFO is unable to stop them from having wait times that are much longer than their execution times. This is made worse by the `sync_channel` limit of 100, which causes the dispatcher to halt when the channel fills with CPU tasks. Second, employees lock a shared Mutex to compete for jobs, and the OS scheduler selects the winner each time. This balances out across a large number of jobs, but in brief studies, scheduling variance alone may cause certain workers to handle more tasks than others. Both problems would be resolved by dividing CPU and IO jobs into two queues with specialized workers.